

ASSIGNMENT 13.3

D.Charmi

2303A51314

Batch – 05

TASK – 01

PROMPT : Remove the repeated logic in the code below and make it more concise and readable by using docstrings.

```
"""print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))"""
```

CODE :

```
#Remove the repeated logic in the code below and make it more concise and
readable by using docstrings.
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
#Refactored Code
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle given its length and
    width.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
```

```

    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")

```

OUTPUT :

```

PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/
ing/13.3.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>

```

EXPLANATION :

The refactored code defines a function `calculate\_rectangle\_properties` that takes the length and width of a rectangle as parameters and returns its area and perimeter. This eliminates the need for repeated calculations and makes the code more concise and readable. The function is then called for each rectangle in the list, and the results are printed in a clear format.

TASK – 02

PROMPT : Optimize the below code by removing the nested if statements and using a single if statement with multiple conditions.

```
names = ["Alice", "Bob", "Charlie", "David"]
```

```
search_names = ["Charlie", "Eve", "Bob"]
```

```
for s in search_names:
```

```
    found = False
```

```
    for n in names:
```

```
        if s == n:
```

```
found = True
```

```
if found:
```

```
    print(f"{s} is in the list")
```

```
else:
```

```
    print(f"{s} is not in the list")'''
```

CODE :

```
#Optimize the beow code by removing the nested if statements and using a
single if statement with multiple conditions.
'''names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
            if found:
                print(f"{s} is in the list")
            else:
                print(f"{s} is not in the list")'''
#Refactored code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    if s in names:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

OUTPUT :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:\Users\devis\OneDrive\Desktop\AI Assisted Coding\3.py
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

EXPLANATION :

The refactored code eliminates the need for a nested loop and a boolean flag to track whether a name was found. Instead, it uses the `in` keyword to directly

check if each search name exists in the list of names. This results in cleaner and more efficient code, as it reduces the number of iterations and simplifies the logic. The performance improvement is significant, especially as the size of the lists increases, since the original code has a time complexity of $O(n*m)$ (where n is the length of `search\_names` and m is the length of `names`), while the refactored code has a time complexity of $O(n)$ due to the use of a set or list lookup.

TASK – 03

PROMPT : Generate a modularized version of the code given below that uses functions to perform the same tasks.

```
'''price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)'''
```

CODE :

```
#Generate a modularized version of the code given below that uses functions to
perform the same tasks.
'''price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)'''

def calculate_total_price(price):
    tax = price * 0.18
    total = price + tax
    return total
```

```
price1 = 250
total1 = calculate_total_price(price1)
print("Total Price:", total1)
price2 = 500
total2 = calculate_total_price(price2)
print("Total Price:", total2)
```

OUTPUT :

```
PS C:\Users\devis\OneDrive
ing/13.3.py"
Total Price: 295.0
Total Price: 590.0
PS C:\Users\devis\OneDrive
```

EXPLANATION :

In the original code, the calculation of the total price including tax is repeated for each price value. By creating a function `calculate\_total\_price`, we can encapsulate the logic for calculating the total price, making the code more modular and reusable. This way, we can simply call the function with different price values without duplicating the calculation logic, improving readability and maintainability.

TASK – 04

PROMPT : Refactor the below code by using proper names for constants. Also, add comments to explain the code.

```
'''print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)'''
```

CODE :

```
# Refactor the below code by using proper names for constants. Also, add
comments to explain the code.
'''print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)'''

# Define constants for the value of pi and the radius of the circle
PI = 3.14159
RADIUS = 7
# Calculate the area of the circle using the formula: Area =  $\pi * r^2$ 
```

```

area_of_circle = PI * (RADIUS ** 2)
# Calculate the circumference of the circle using the formula: Circumference =
2 * π * r
circumference_of_circle = 2 * PI * RADIUS
# Print the area and circumference of the circle
print("Area of Circle:", area_of_circle)
print("Circumference of Circle:", circumference_of_circle)

```

OUTPUT :

```

PS C:\Users\devis\OneDrive\Desktop\AI Assist
ing\13.3.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\devis\OneDrive\Desktop\AI Assist

```

EXPLANATION :

In this refactored code, we have defined constants for the value of pi (PI) and the radius of the circle (RADIUS) to improve readability and maintainability. We then calculate the area and circumference of the circle using the appropriate formulas and store the results in variables (area_of_circle and circumference_of_circle) before printing them. This makes the code more organized and easier to understand.

TASK – 05

PROMPT : Refactor the below code by using proper variable names, adding comments, and improving readability without changing the functionality.

```

'''a = 10
b = 20
c = a * b / 2
print(c)'''

```

CODE :

```

#Refactor the below code by using proper variable names, adding comments, and
improving readability without changing the functionality.

'''a = 10
b = 20

```

```
c = a * b / 2
print(c)'''
# Refactored Code
# Calculate the area of a triangle given its base and height
base = 10 # Base of the triangle
height = 20 # Height of the triangle
area_of_triangle = (base * height) / 2 # Area formula: (base * height) / 2
print(area_of_triangle) # Output the area of the triangle
```

OUTPUT :

```
100.0
PS C:\Users\devis\OneD
ing/13.3.py"
100.0
```

EXPLANATION :

In the refactored code, I have renamed the variables to be more descriptive of their purpose. I have also added comments to explain what each variable represents and the formula being used to calculate the area of the triangle. This improves readability and makes it easier for others to understand the code.